

The monad **State**

Luca Roversi

December 11, 2025

Learning outcomes

- The monad `State` is a (without/with `runState`)
- Interpreter with expressions depending on variables:

- `Terms` such that:

```
data Terms a where
```

```
  Tvar :: String -> Terms a
```

```
  ...
```

```
  Tquo :: Terms a -> Terms a -> Terms a
```

- The interpreter has type

```
Terms BN -> State [(String,BN)] ErrMsg (Result BN)
```

The monad `State` (without `runState`)

- `Arithmetic.Interpreters.Monadic.State.hs`
- `Arithmetic.Interpreters.Monadic.StateIsMonad.hs`

The monad `State` (with `runState`)

-- A stateful computation: takes a state and returns a value and new state

```
newtype State s a = State { runState :: s -> (a, s) }
```

```
instance Monad (State s) where
```

-- Injects a pure value into the monadic context

```
return :: a -> State s a
```

```
return = pure
```

-- Bind operator: sequences two computations

```
(>>=) :: State m a -> (a -> State m b) -> State m b
```

```
(>>=) f g =
```

```
State $ \m ->
```

-- Step 1: run the first computation with initial state

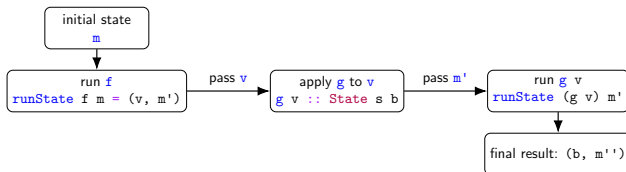
```
let (v, m') = runState f m
```

-- Step 2: apply function g to result v, yielding a new computation

-- Step 3: run that new computation with updated state m'

```
in runState (g v) m'
```

Computation flow



Interpreter for expressions with variables

- `Arithmetic.Expressions.Vars.hs`
- `Arithmetic.Interpreters.Monad.Vars.Operations.hs`
- `Arithmetic.Interpreters.Monad.Vars.Eval.hs`